REPUBLIC OF THE PHILIPPINES
BICOL UNIVERSITY
POLANGUI
COMPUTER STUDIES DEPARTMENT
Polangui, Albay

ISO 9001: 2015
SOCOTEC SCP000722Q

BAGONG PILIPINAS

WEB SYSTEMS AND TECHNOLOGIES

Phase 2 – Backend Development and API Integration

LAB REPORT

Uniformity: An E-Commerce platform for Pre-Loved School and Professional Uniforms

GROUP NAME: TechNic **Course & Block:** BSIT-2A		
MEMBERS	**ROLE**	**SIGNATURE**
Cipriano, Mcxyron B.	*Back-End Developer*	
Hernandez, Kurt Jushua S.	*Database Manager/Tester & Debugger*	
Registrado, Mharie Franz	*Tester & Debugger*	
Red, Paul Orlando B.	*Project, Database, GitHub Manager & Documentation Officer*	
Romano, Jay L.	*Front-End Developer*	

Submitted to:

Guillermo V. Red, Jr., DIT

Assistant Professor IV

I. Overview of Backend Setup

This laboratory report documents the Phase 2 backend development of our project Uniformity, a web-based e-commerce platform for buying and selling pre-loved school and professional uniforms. The backend was built using Node.js and Express, with MongoDB as the database and Mongoose as the Object Data Modeling (ODM) library.

The backend follows a modular and scalable MVC-inspired architecture, separating concerns into distinct folders: models, controllers, routes, and config. This structure ensures maintainability as the project grows across the remaining phases.

Tech Stack Used:

- **Node.js —** JavaScript runtime for the server
- **Express —** Web framework for building RESTful APIs
- **MongoDB —** NoSQL database for storing application data
- **Mongoose —** ODM library for schema definition and database interaction
- **dotenv —** Environment variable management
- **cors —** Cross-Origin Resource Sharing middleware
- **nodemon —** Auto-restart server during development
- **bcryptjs —** Password hashing for secure user registration and login
- **jsonwebtoken —** JWT generation and verification for authentication
- **multer —** File and image upload handling
- **cloudinary API / multer-storage-cloudinary —** Cloud image storage for product listings

II. Code Snippets

A. server.js - Main Server Setup

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const connectDB = require('./config/db');

const app = express();

// Connect to Database
connectDB();
```

```
// Middleware
app.use(cors());
app.use(express.json());

// Define Routes
app.use('/api/auth', require('./routes/authRoutes')); // New Auth Routes
app.use('/api/users', require('./routes/userRoutes'));
app.use('/api/categories', require('./routes/categoryRoutes'));
app.use('/api/products', require('./routes/productRoutes'));
app.use('/api/cart', require('./routes/cartRoutes'));
app.use('/api/orders', require('./routes/orderRoutes'));

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

B. Sample Model - Product.js

The Product model defines the schema for uniform listings. It references both the User (as seller) and Category collections using ObjectId foreign keys, reflecting the ERD designed in Phase 1.

```
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: { type: String, required: true },
  size: { type: String, required: true },
  quantity: { type: Number, required: true },
  price: { type: Number, required: true },
  description: { type: String },
  image_url: { type: String },
  seller_id: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  category_id: { type: mongoose.Schema.Types.ObjectId, ref: 'Category', required: true },
  created_at: { type: Date, default: Date.now }
});

module.exports = mongoose.model('Product', productSchema);
```

C. Sample Route - productRoutes.js

Routes are kept thin — they only define the HTTP method and path, then delegate logic to the controller. This separation of concerns makes the code easier to maintain and test.

```python
const express = require('express');
const router = express.Router();
const {
    createProduct,
    getAllProducts,
    getProductById,
    updateProduct,
    deleteProduct
} = require('../controllers/productController');

// new middlewares
const auth = require('../middleware/auth');
const { upload } = require('../config/cloudinary');

// @route   POST /api/products
router.post('/', [auth, upload.single('image')], createProduct);

// Public routes
router.get('/', getAllProducts);
router.get('/:id', getProductById);

// Protected update and delete
router.put('/:id', auth, updateProduct);
router.delete('/:id', auth, deleteProduct);

module.exports = router;
```

D. Sample Controller - productController.js

Controllers contain the logic for each route. The getAllProducts controller supports optional query parameters for filtering by category or seller, enabling the browse and seller dashboard features planned for later phases.

```python
const Product = require('../models/Product');

// @desc    Create a new product listing
exports.createProduct = async (req, res) => {
    try {
        // Build product object
        const productData = {
            ...req.body,
            // The seller_id is taken from the token
            seller_id: req.user.id,
            // The image_url is the path provided by Cloudinary
            image_url: req.file ? req.file.path : ''
        };

        const product = new Product(productData);
        await product.save();
        res.status(201).json(product);
    } catch (err) {
        console.error(err.message);
        res.status(400).json({ error: err.message });
    }
};
```

```python
exports.getAllProducts = async (req, res) => {
    try {
        const filter = {};
        if (req.query.category) filter.category_id = req.query.category;
        if (req.query.seller) filter.seller_id = req.query.seller;

        const products = await Product.find(filter)
            .populate('seller_id', 'name email')
            .populate('category_id', 'name');
        res.json(products);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};
```

```python
exports.getProductById = async (req, res) => {
    try {
        const product = await Product.findById(req.params.id)
            .populate('seller_id', 'name email')
            .populate('category_id', 'name');

        if (!product) return res.status(404).json({ error: 'Product not found' });
        res.json(product);
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};
```

```
exports.updateProduct = async (req, res) => {
    try {
        // Ensure user is the owner before updating (Optional but recommended)
        const product = await Product.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true }
        );
        res.json(product);
    } catch (err) {
        res.status(400).json({ error: err.message });
    }
};

exports.deleteProduct = async (req, res) => {
    try {
        await Product.findByIdAndDelete(req.params.id);
        res.json({ message: 'Product deleted successfully' });
    } catch (err) {
        res.status(500).json({ error: err.message });
    }
};
```

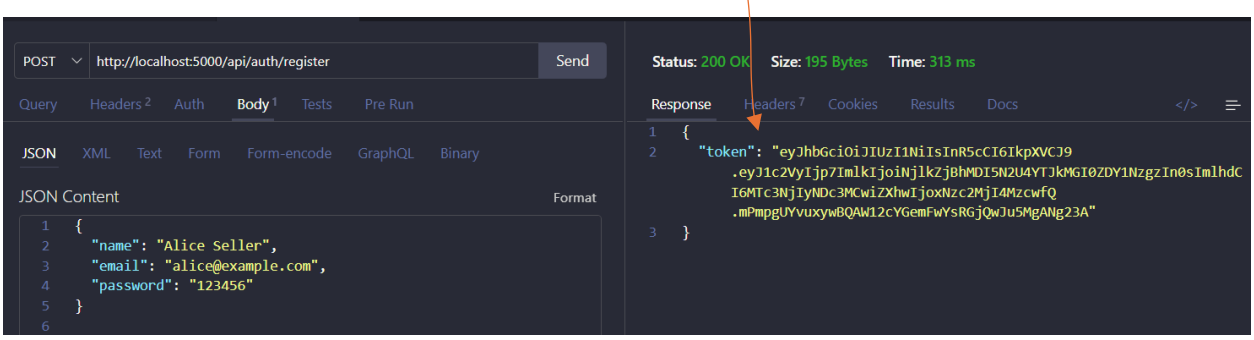
III. API Testing

All routes were tested using Thunder Client inside VS Code. Below are the documented test cases for each resource.

A. Auth

authRoutes.js - /register

Returned token



Hashed password

```
_id: ObjectId('69df0a0297e8a2d0b4d65783')
name : "Alice Seller"
email : "alice@example.com"
password : "$2b$10$T4G0Fmreho9Y0LmC942va.GKjj/l7StMwRObCuo2/eW5.xMmVl2T."
created_at : 2026-04-15T03:46:10.118+00:00
__v : 0
```

authRoutes.js - /login

POST http://localhost:5000/api/auth/login Send

Query Headers 2 Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "email": "alice@example.com",
  "password": "123456"
}
```

Status: 200 OK Size: 195 Bytes Time: 195 ms

Response Headers 7 Cookies Results Docs


```
{
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
      .eyJ1c2VyIjp7ImlkIjoiNjlkZjBhMDI5N2U4YTJkMGI0ZDY1NzgzIn0sImlhdC
      I6MTc3NjIyNTczMSwiZXhwIjoxNzc2MjI5MzMxfQ
      .QeznRo9GgkWhViWcP1ifpy2qOdvdCHzELmaVYT6Z8CI"
}
```


B. Users

userRoutes.js - /users (POST)

POST http://localhost:5000/api/users Send

Query Headers Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "Bob Buyer",
  "email": "bob@example.com",
  "password": "123456"
}
```

Status: 201 Created Size: 147 Bytes Time: 91 ms

Response Headers 7 Cookies Results Docs


```
{
  "name": "Bob Buyer",
  "email": "bob@example.com",
  "password": "123456",
  "_id": "69df0fe997e8a2d0b4d65784",
  "created_at": "2026-04-15T04:11:21.431Z",
  "__v": 0
}
```


userRoutes.js - /users (GET)

GET http://localhost:5000/api/users Send

Query Headers Auth Body 1 Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 356 Bytes Time: 76 ms

Response Headers 7 Cookies Results Docs


```
[
  {
    "_id": "69df0a0297e8a2d0b4d65783",
    "name": "Alice Seller",
    "email": "alice@example.com",
    "password": "$2b$10$T4G0Fmreho9Y0LmC942va.GKjj/l7StMwRObCuo2
        /eW5.xMmVl2T.",
    "created_at": "2026-04-15T03:46:10.118Z",
    "__v": 0
  },
  {
    "_id": "69df0fe997e8a2d0b4d65784",
    "name": "Bob Buyer",
    "email": "bob@example.com",
    "password": "123456",
    "created_at": "2026-04-15T04:11:21.431Z",
    "__v": 0
  }
```


userRoutes.js - /users/:id (GET)

GET http://localhost:5000/api/users/69df0a0297e8a2d0b4d65783 Send

Query Headers Auth Body 1 Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 206 Bytes Time: 57 ms

Response Headers 7 Cookies Results Docs


```
{
  "_id": "69df0a0297e8a2d0b4d65783",
  "name": "Alice Seller",
  "email": "alice@example.com",
  "password": "$2b$10$T4G0Fmreho9Y0LmC942va.GKjj/l7StMwRObCuo2/eW5
      .xMmVl2T.",
  "created_at": "2026-04-15T03:46:10.118Z",
  "__v": 0
}
```

userRoutes.js - /users/:id (PUT)

PUT http://localhost:5000/api/users/69df0a0297e8a2d0b4d65783 Send

Query Headers Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "Alice Dela Cruz"
}
```

Status: 200 OK Size: 209 Bytes Time: 167 ms

Response Headers 7 Cookies Results Docs

```
{
  "_id": "69df0a0297e8a2d0b4d65783",
  "name": "Alice Dela Cruz",
  "email": "alice@example.com",
  "password": "$2b$10$T4G0Fmreho9Y0LmC942va.GKjj/l7StMwRObCuo2/eW5
      .xMmVlzT.",
  "created_at": "2026-04-15T03:46:10.118Z",
  "__v": 0
}
```

userRoutes.js - /users/:id (DELETE)

DELETE http://localhost:5000/api/users/69df0a0297e8a2d0b4d65783 Send

Query Headers Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

Status: 200 OK Size: 26 Bytes Time: 39 ms

Response Headers 7 Cookies Results Docs

```
{
  "message": "User deleted"
}
```

C. Categories

categoryRoutes.js - /categories (POST)

POST http://localhost:5000/api/categories Send

Query Headers Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
{
  "name": "School Uniform"
}
```

Status: 201 Created Size: 66 Bytes Time: 36 ms

Response Headers 7 Cookies Results Docs

```
{
  "name": "School Uniform",
  "_id": "69df129a97e8a2d0b4d65786",
  "__v": 0
}
```

categoryRoutes.js - /categories (GET)

GET http://localhost:5000/api/categories Send

Query Headers Auth Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

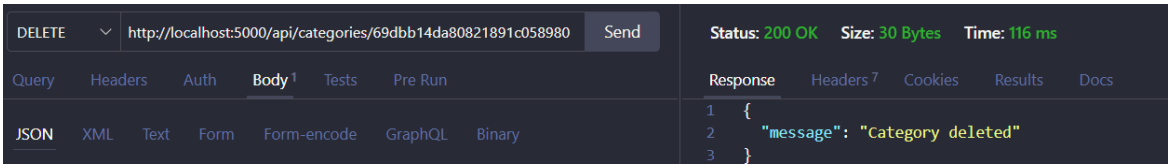
```
{
  "name": "School Uniform"
}
```

Status: 200 OK Size: 141 Bytes Time: 156 ms

Response Headers 7 Cookies Results Docs

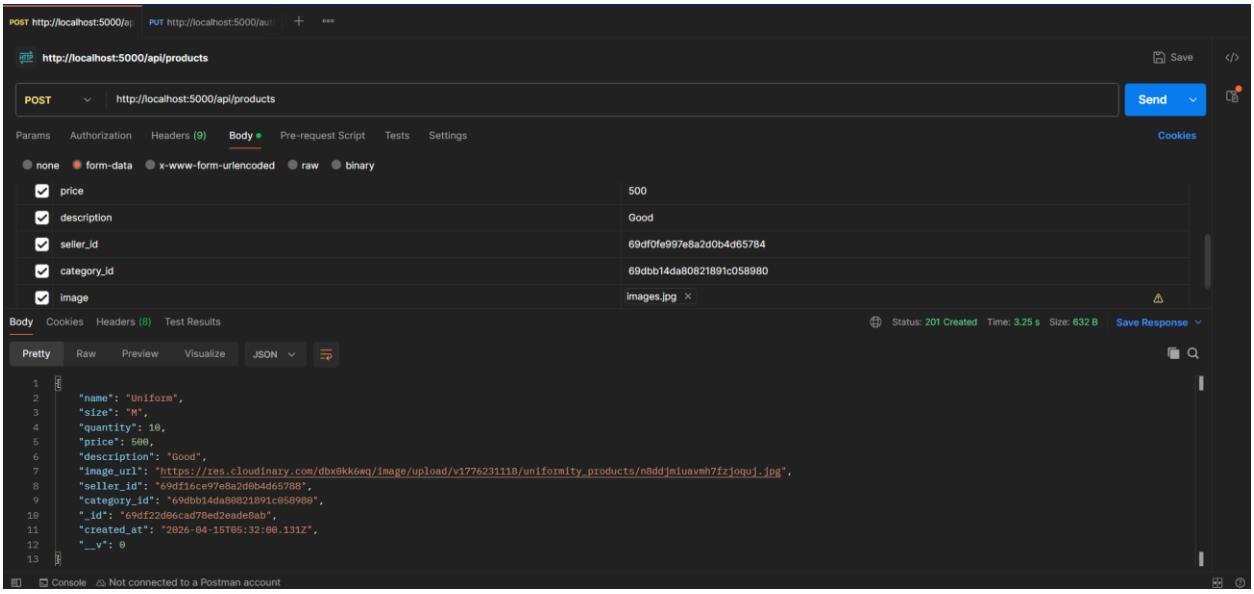
```
[
  {
    "_id": "69dbb14da80821891c058980",
    "name": "Professional Uniform",
    "__v": 0
  },
  {
    "_id": "69df129a97e8a2d0b4d65786",
    "name": "School Uniform",
    "__v": 0
  }
]
```

categoryRoutes.js - /categories/:id (DELETE)



D. Products and Cloudinary

productRoutes.js - /products (POST)



Since thunder client doesn't allow image upload in free tier, I use postman instead.

Also, the product route are protected so it needs a token that is obtain by logging in or registering

The reply is status 201 which means it's posted successfully with returned image URL from the cloudinary API.


```
"name": "Uniform",
"size": "M",
"quantity": 10,
"price": 500,
"description": "Good",
"image_url": "https://res.cloudinary.com/dbx0kk6wq/image/upload/v1776231118/uniformity_products/n8ddjmiuavmh7fzjoquj.jpg",
"seller_id": "69df16ce97e8a2d0b4d65788",
"category_id": "69dbb14da80821891c058980",
"_id": "69df22d06cad78ed2eade8ab",
"created_at": "2026-04-15T05:32:00.131Z",
"__v": 0
```

productRoutes.js - /products (GET)

GET http://localhost:5000/api/products Send

Query Headers 1 Auth 1 Body 1 Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 1.23 KB Time: 218 ms

Response Headers 7 Cookies Results Docs


```
    "__v": 0
  },
  {
    "_id": "69df22d06cad78ed2eade8ab",
    "name": "Uniform",
    "size": "M",
    "quantity": 10,
    "price": 500,
    "description": "Good",
    "image_url": "https://res.cloudinary.com/dbx0kk6wq/image/upload
        /v1776231118/uniformity_products/n8ddjmiuavmh7fzjoquj.jpg",
    "seller_id": {
      "_id": "69df16ce97e8a2d0b4d65788",
      "name": "Alice Seller",
      "email": "alice@example.com"
    },
    "category_id": null,
    "created_at": "2026-04-15T05:32:00.131Z",
    "__v": 0
```


productRoutes.js - /products/:id (GET)

GET http://localhost:5000/api/products/69df22d06cad78ed2eade8ab Send

Query Headers 1 Auth 1 Body 1 Tests Pre Run

Query Parameters

parameter value

Status: 200 OK Size: 394 Bytes Time: 70 ms

Response Headers 7 Cookies Results Docs


```
{
  "_id": "69df22d06cad78ed2eade8ab",
  "name": "Uniform",
  "size": "M",
  "quantity": 10,
  "price": 500,
  "description": "Good",
  "image_url": "https://res.cloudinary.com/dbx0kk6wq/image/upload
      /v1776231118/uniformity_products/n8ddjmiuavmh7fzjoquj.jpg",
  "seller_id": {
    "_id": "69df16ce97e8a2d0b4d65788",
    "name": "Alice Seller",
    "email": "alice@example.com"
  },
  "category_id": null,
  "created_at": "2026-04-15T05:32:00.131Z",
  "__v": 0
}
```


productRoutes.js - /products/:id (PUT) in editing products

PUT http://localhost:5000/api/products/69df22d06cad78ed2eade8ab Send

Query Headers 1 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format


```
{
  "price": 300,
  "quantity": 2
}
```


Status: 200 OK Size: 357 Bytes Time: 51 ms

Response Headers 7 Cookies Results Docs


```
{
  "_id": "69df22d06cad78ed2eade8ab",
  "name": "Uniform",
  "size": "M",
  "quantity": 2,
  "price": 300,
  "description": "Good",
  "image_url": "https://res.cloudinary.com/dbx0kk6wq/image/upload
      /v1776231118/uniformity_products/n8ddjmiuavmh7fzjoquj.jpg",
  "seller_id": "69df16ce97e8a2d0b4d65788",
  "category_id": "69dbb14da80821891c058980",
  "created_at": "2026-04-15T05:32:00.131Z",
  "__v": 0
}
```


productRoutes.js - /products/:id (DELETE)

DELETE http://localhost:5000/api/products/69dbb2e5714c1ace6241348c Send

Query Headers 1 Auth 1 Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

Status: 200 OK Size: 42 Bytes Time: 45 ms

Response Headers 7 Cookies Results Docs


```
{
  "message": "Product deleted successfully"
}
```

E. Cart

cartRoutes.js - /cart/:userId (GET)

GET http://localhost:5000/api/cart/69df0fe997e8a2d0b4d65784 Send

Query Headers 1 Auth 1 Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1
```

Status: 200 OK Size: 139 Bytes Time: 119 ms

Response Headers 7 Cookies Results Docs

```
1 {
2   "cart": {
3     "user_id": "69df0fe997e8a2d0b4d65784",
4     "_id": "69df2aca6cad78ed2eade8ac",
5     "created_at": "2026-04-15T06:06:02.738Z",
6     "__v": 0
7   },
8   "items": []
9 }
```

cartRoutes.js - /cart/add (POST)

POST http://localhost:5000/api/cart/add Send

Query Headers 1 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1 {
2   "cart_id": "69df2aca6cad78ed2eade8ac",
3   "product_id": "69df22d06cad78ed2eade8ab",
4   "quantity": 1
5 }
6
```

Status: 201 Created Size: 132 Bytes Time: 99 ms

Response Headers 7 Cookies Results Docs

```
1 {
2   "cart_id": "69df2aca6cad78ed2eade8ac",
3   "product_id": "69df22d06cad78ed2eade8ab",
4   "quantity": 1,
5   "_id": "69df2b3c6cad78ed2eade8ad",
6   "__v": 0
7 }
```

cartRoutes.js - /cart/item/cart_item_id (DELETE)

DELETE http://localhost:5000/api/cart/item/69df2b3c6cad78ed2eade8ad Send

Query Headers 1 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

Status: 200 OK Size: 36 Bytes Time: 61 ms

Response Headers 7 Cookies Results

```
1 {
2   "message": "Item removed from cart"
3 }
```

orderRoutes.js - /orders (POST)

POST http://localhost:5000/api/orders Send

Query Headers 1 Auth 1 Body 1 Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

```
1  {
2    "buyer_id": "69df0fe997e8a2d0b4d65784",
3    "delivery_address": "Manila Sample Address",
4    "total_amount": 350,
5    "items": [
6      {
7        "product_id": "69df22d06cad78ed2eade8ab",
8        "seller_id": "69df16ce97e8a2d0b4d65788",
9        "quantity": 1,
10       "price": 350
11     }
12   ]
13 }
14
```

Status: 201 Created Size: 430 Bytes Time: 184 ms

Response Headers 7 Cookies Results Docs

```
1  {
2    "order": {
3      "buyer_id": "69df0fe997e8a2d0b4d65784",
4      "delivery_address": "Manila Sample Address",
5      "total_amount": 350,
6      "status": "pending",
7      "_id": "69df2d286cad78ed2eade8ae",
8      "created_at": "2026-04-15T06:16:08.030Z",
9      "__v": 0
10   },
11   "orderItems": [
12     {
13       "order_id": "69df2d286cad78ed2eade8ae",
14       "product_id": "69df22d06cad78ed2eade8ab",
15       "seller_id": "69df16ce97e8a2d0b4d65788",
16       "quantity": 1,
17       "price": 350,
18       "status": "pending",
19       "_id": "69df2d286cad78ed2eade8af"
```

orderRoutes.js - /orders/buyer/buyer_id (GET)

GET http://localhost:5000/api/orders/buyer/69d8f44a0a10e5b9c1ecf592 Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

Status: 200 OK Size: 508 Bytes Time: 59 ms

Response Headers 7 Cookies Results Docs


```
[
  {
    "_id": "69d8f4810a10e5b9c1ecf59c",
    "buyer_id": "69d8f44a0a10e5b9c1ecf592",
    "delivery_address": "sefsfsf, polangui, albay, 4506",
    "total_amount": 1000,
    "status": "delivered",
    "created_at": "2026-04-10T13:00:49.345Z",
    "updated_at": "2026-04-10T13:00:49.345Z",
    "__v": 0
```


orderRoutes.js - /orders/seller/seller_id (GET)

GET http://localhost:5000/api/orders/seller/69df16ce97e8a2d0b4d65788 Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

Status: 200 OK Size: 538 Bytes Time: 88 ms

Response Headers 7 Cookies Results Docs


```
[
  {
    "_id": "69df2d286cad78ed2eade8af",
    "order_id": {
      "_id": "69df2d286cad78ed2eade8ae",
      "buyer_id": "69df0fe997e8a2d0b4d65784",
      "delivery_address": "Manila Sample Address",
      "total_amount": 350,
      "status": "pending",
      "created_at": "2026-04-15T06:16:08.030Z",
      "__v": 0
    },
```


orderRoutes.js - /orders/seller/seller_id (GET)

GET http://localhost:5000/api/orders/seller/69df16ce97e8a2d0b4d65788 Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

JSON Content Format

Status: 200 OK Size: 538 Bytes Time: 88 ms

Response Headers 7 Cookies Results Docs

```
[
  {
    "_id": "69df2d286cad78ed2eade8af",
    "order_id": {
      "_id": "69df2d286cad78ed2eade8ae",
      "buyer_id": "69df0fe997e8a2d0b4d65784",
      "delivery_address": "Manila Sample Address",
      "total_amount": 350,
      "status": "pending",
      "created_at": "2026-04-15T06:16:08.030Z",
      "__v": 0
    },
```


orderRoutes.js - /orders/item/order_item_id (PATCH)

PATCH http://localhost:5000/api/orders/item/69df2d286cad78ed2eade8af/fulf Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

Status: 200 OK Size: 28 Bytes Time: 152 ms

Response Headers 7 Cookies Results Docs


```
{
  "message": "Item fulfilled"
}
```


orderRoutes.js - /orders/item/order_id/deliver (PATCH)

PATCH http://localhost:5000/api/orders/69d8fe57b9b5a9030e703931/deliver Send

Query Headers 2 Auth Body Tests Pre Run

JSON XML Text Form Form-encode GraphQL Binary

Status: 200 OK Size: 39 Bytes Time: 90 ms

Response Headers 7 Cookies Results Docs


```
{
  "message": "Order marked as delivered"
```

IV. Github Commits

Add Mongoose models for Cart, CartItem, Category, Order, OrderItem, Product, and User
red5458 committed 3 days ago

Merge pull request #1 from red5458/database-manager
red5458 authored 3 days ago

Add database connection setup using mongoose
red5458 committed 3 days ago

Add seed script for categories in the database
red5458 committed 3 days ago

Note: The backend commits are arranged from top (oldest) up to bottom (latest)

Add controllers, routes, and server setup #8

Merged red5458 merged 3 commits into `main` from `backend` 3 days ago

Conversation 1 | Commits 3 | Checks 0 | Files changed 12

Commits on Apr 12, 2026

Added controllers
Xye1110 committed 3 days ago

Added routes
Xye1110 committed 3 days ago

Added server.js
Xye1110 committed 3 days ago

Add README.md with project overview, tech stack, folder structure, models, connection setup, and API routes
red5458 committed 3 days ago

Add initial package.json for backend setup with dependencies & fix orderController not providing orderItems_id
red5458 committed 3 days ago

fix upload dependency conflict with Cloudinary
red5458 committed 3 hours ago

add .npmrc to handle legacy peer dependencies
red5458 committed 3 hours ago

Updated the Project README with the banding/logo
red5458 committed 4 hours ago

Updated the Readme in the backend folder
red5458 committed 4 hours ago

Integrate Cloudinary API and Multer for product image uploads. And add cloudinary.js in config folder and update Product controller & routes js
red5458 committed 5 hours ago

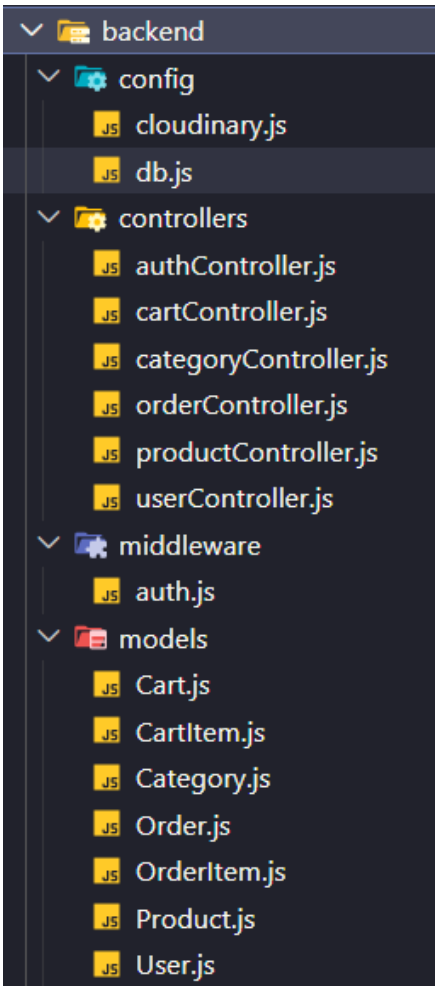
fix: connect auth routes to main server
red5458 committed 5 hours ago

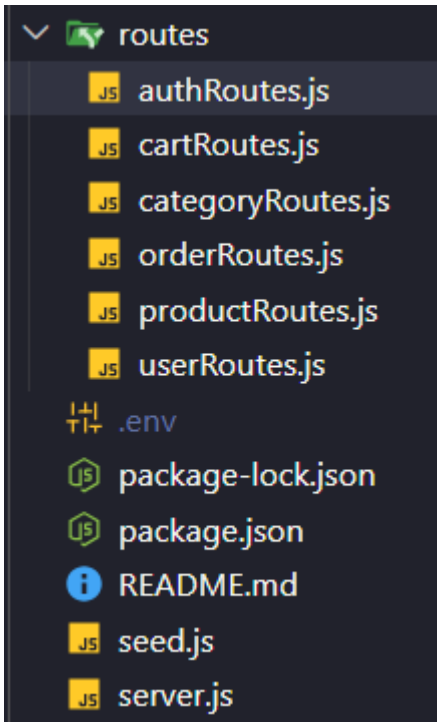
implement auth controller with bcrypt hashing and JWT generation
red5458 committed 5 hours ago

feat: add JWT authentication middleware
red5458 committed 5 hours ago

Updated the package json for the added API dependencies
red5458 committed 6 hours ago

V. Backend Folder Structure





V. Group Member Contributions

Member	Role	Contribution
Cipriano, Mcxyron B.	Back-End Developer	Developed all Express routes and controllers. Built the order fulfillment logic including the auto-update of ORDERS status when all ORDER_ITEMS are fulfilled.
Hernandez, Kurt Jushua S.	Database Manager / Tester & Debugger	Designed and implemented all Mongoose models based on the finalized updated ERD. Set up the MongoDB connection via Mongoose and managed the database configuration.

Member	Role	Contribution
Registrado, Mharie Franz	Tester & Debugger	Conducted API testing using Thunder Client for all routes. Documented test cases, identified and reported bugs, and also helps in making this lab report in this Phase.
Red, Paul Orlando B.	Project, Database, GitHub Manager & Documentation Officer	Managed the GitHub repository structure and commit history. Wrote the backend README.md and this lab report. Coordinated task assignments across the group.
Romano, Jay L.	Front-End Developer	Assisted in reviewing the API endpoints and response structures to ensure alignment with the planned frontend UI for Phase 3.

Prepared by:

Mcxyron B. Cipriano
Back-End Developer

Kurt Jushua S. Hernandez
Database Manager/Tester & Debugger

Mharie Franz Registrado
Tester & Debugger

Paul Orlando B. Red
Project, Database, GitHub Manager & Documentation Officer

Jay L. Romano
Front-End Developer